Numerical Stability: A Toy Problem

Rodrigo Gonzalez

June 27, 2026

Abstract

Here we present a toy problem that illustrates the main ideas behind numerical stability of weak forms for advection-diffusion PDEs.

1 Introduction

Consider the following steady-state advection-diffusion problem.

$$du_{xx} + au_x = f(x)$$

with $x \in [0, 1]$ and homogeneous boundary conditions. It is well known that a pure Galerkin approach to numerically solving the problem will sometimes lead to bad results. This is especially true in the advection-dominated limit. In fact, if we define the *peclet number* as the ratio between the advective and diffusive transport rates, i.e. $\mathbf{Pe} = a/d$, then the Galerkin approach presents numerical inestabilities as **Pe** goes to infinity.

Here we present a toy problem using 2 by 2 matrices that illustrates the mechanisms by which these numerical inestabilities appear, as well as the main ideas behind different stabilization techniques. This toy example will hopefully clarify some aspects of the "real-world" techniques by providing an intuition which is easier to convey this context.

2 The Exact and the Numerical Problems

We may abstractly rewrite the *exact problem* as:

Find $u(x)$ (living in some **infinite-dimensional** functional space, call it $\mathbf{V}$) such that

$$Lu = f$$

Where L is the differential operator presented before and f is some given function.

Evidently, in a computer it is impossible to solve this problem exactly: the solution lives in an infinite-dimensional space, and no computer will be able to capture infinity. We therefore define the *numerical problem* as:

Find $u_h(x)$ (living in some **finite-dimensional** functional space, call it $\mathbf{V_h}$) such that

$$Lu_h = f$$

Where L and f are defined as before. Now, a computer representation of u_h becomes available, since by construction, only a finite amount of data is required to represent it. Evidently, in general we will not have that $u_h = u$, we have introduced a *numerical error* by solving an approximation of our original problem.

We may simply define numerical stability as some kind of continuity between the numerical and exact solution, and the two problems. We would like that going from the exact to the numerical only costs us an acceptable amount of error: the "distance" between u and u_h should not be unbounded or uncontrolled.

3 The Weak Form

We will continue discussing the meaning of numerical stability in the following sections, but for now let us review how to obtain the weak form of the equation.

In the context of PDEs, a weak form is obtained by multiplying by a test function and integrating. This is equivalent to taking the L^2 inner product. Abstractly, if the strong form of the PDE looks like

$$Lu = f$$

Then, the weak form looks like

$$\big(Lu, \phi\big) = \big(f, \phi\big)$$

Where $(\cdot, \cdot)$ represents said inner product. Note that there is nothing special about considering functions, differential operators, and integrals - the same concept can be used in matrices, and system of equations in R^n. Indeed, now consider an $n \times n$ matrix A, b a given vector, and the following system

$$Ax = b$$

where x is the unknown vector to be determined. Then we may produce the "weak form" by taking the "dot product" of this equation with any other vector y:

$$y^T Ax = y^T b$$

4 The Toy Problem

Consider solving the following system of equations:

$$Ax = b$$

Where the matrix is given by

$$A = d \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + a \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

and a and d are real numbers and d is assumed to be positive. Note that this matrix has determinant $a^2 + d^2$ so it is invertible as long as both coefficients are not both zero. To obtain a right hand side assume that

$$x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Then we have:

$$b = \begin{bmatrix} d + a \\ d - a \end{bmatrix}$$

Suppose now that we ignore what the solution x is and we are trying to solve the system. Following the previous section, a "weak formulation" approach to this would require us to choose a suitable vector space V, and find the $x \in V$ such that for all $y \in V$ we have

$$y^T Ax = y^T b$$

Indeed, if we take $V = \mathbb{R}^2$, and take the canonical basis $\{e_x, e_y\}$, expressing x as a linear combination of these vectors, and iterating y over these two leads precisely to the original system and the correct solution.

5 Numerical Instability

In numerical analysis, we can not often access the full vector space V and instead are left to solve the problem in a (finite-dimensional) vector subspace $V_h. \subset V$. Mimicking this, suppose that for our toy problem we are confined to the subspace $V_h = \{e_x\}$. Then, for the numerical problem, we are looking for $x_h \in V_h$ such that for all $y \in V_h$, we have:

$$y^T A x_h = y^T b$$

We take $x_h = \chi e_x$ and $y = e_x$, we obtain only one equation out of our numerical problem:

$$d\chi = d + a$$

were we solve for the coefficient χ to obtain that

$$x_h = \chi e_x = 1 + a/d \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} a/d \\ 0 \end{bmatrix}$$

How much error have we introduced by solving our problem not in the full space, but only in a lower-dimensional space? No matter how hard we tried, we were going to make some mistake because our true solution x is not in V_h - it is not in SPAN$\{e_x\}$. However, we can compare x_h to the closest (in which sense? More on this when we discuss least-squares) element in V_h. Evidently, this is given by the L^2 projection of x into the subspace:

$$Px = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

which coincidentally coincides with e_x. Then our error is given by

$$e_h = Px - x_h = \begin{bmatrix} a/d \\ 0 \end{bmatrix}$$

Or in norm

$$||e_h|| = |a/d|$$

which resembles closely the Peclet number discussed before. Notice that this error is 0 if $a = 0$, and increases as d grows relative to a and goes to infinity as d approaches 0.

This is alarming! It seems that our approach approximating the solution on a lower-dimensional space is very sensitive to the parameters of the problem, giving completely opposite results in different cases.

6 Analysis: Loss of Information

We would like to have a better understanding of this phenomenon. While there are several ways of explaining this, we point out that the discrete problem, by constraining the search of a solution to a smaller space, will necessarily leave out some information from the original problem. This ignored information is correlated and central to the numerical error that we are producing. Moreover, it seems that the more information is ignored, the bigger the error that is introduced, leading to numerical instabilities.

To make this more concrete, notice again that our weak form $y^T A x_h = y^T b$ will completely recover the original system of equations if y and x_h are both allowed to be any vector in R^2. However, the numerical problem constrains this vector to smaller subspace leading to a right-hand side of the form

$$e_x^T A e_x$$

Thus producing a new operator acting on the lower-dimensional space. A "discrete operator" if you will. How does this operator look like in the lower-dimensional space? For the matrix A, this operation simply extracts a diagonal element of the matrix, in our case, the first one. broken into its two parts, we obtain:

$$e_x^T \begin{bmatrix} d & 0 \\ 0 & d \end{bmatrix} e_x = d$$

and

$$e_x^T \begin{bmatrix} 0 & a \\ -a & 0 \end{bmatrix} e_x = 0$$

As we can see, the first part of the operator, the symmetric part associated to the coefficient d retains its original purpose: it stretches a vector by a factor of d. In this way there is a preservation of information: the operator went from being d multiplied by the identity in $\mathbb{R}^2$ to being d multiplied by the identity in $\mathbb{R}^1$. On the other hand, the anti-symmetric part associated to the coefficient a completely morphed into something completely different. In fact, all of its information was lost. The operator went from being a multiplied by a rotation in $\mathbb{R}^2$, to just being the zero operator in $\mathbb{R}^1$.

Through this we see that there is a multiplicity, a lack of "one-to-one-ness" in this process. Indeed, by choosing a arbitrarily, we can produce an infinite number of 2 by 2 matrices $\tilde{A}$ will lead to the same discrete one-dimensional operator through the weak formulation that we have described before. In fact, we see that we are equivalently solving with $\tilde{A} = dI$, which is of course a different problem. The information carried by a has become irrelevant in the lower-dimensional problem.

In this way we can understand where the numerical instability and the role of the Peclet number come from. The discrete problem ignores the information from the anti-symmetric part associated with the a coefficient. If the information carried by this part is small compared to the information carried by the other symmetric part (the one associated with the d coefficient) then little is lost and we do not incur in too much error. This is the case of a low Peclet number. However, on the other hand, if the anti-symmetric part is big compared to the symmetric part, we ignore a huge amount of the information carried by the original problem, thus leading to bad results and a big error. This is the case of a large Peclet number.

7 Coercivity and Geometry

Based on a previous analysis, we see that the quantity

$$(Ax, x) \text{ or } x^T Ax$$

appears to be connected to the numerical stability the weak form. Particularly, we note that for the symmetric part, this quantity equaled d, a positive constant, whereas for the anti-symmetric part it equaled 0. More generally, it seems that we would like our operators to be **coercive** in order to enjoy good stability properties.

An operator $A : V \to V$, where V is a inner product linear space space, is said to be coercive if there exists a positive constant c such that for all $x \in V$ we have

$$c||x||^2 \leq (Ax, x)$$

Note that this is a notion of growth, we would like our operator to grow at least quadratically, to be bounded below by the parabola $c||x||^2$. Another interpretation of the geometry of coercivity is as follows. Take any vector x in the unit sphere of $\mathbb{R}^n$. Then, apply the operator to obtain Ax. How far away has Ax gone from its starting point x? A way to measure this change is by taking the inner product of this two. In Euclidean spaces this is related to measuring the angle between them. Or alternatively, (Ax, x) can be thought of the length of the projection of Ax onto the original x. Coercivity ensures that this length has a minimum positive value. Additonally, it also ensures that the image of A is contained by a "cone", where the axis of this cone is always "normalized" to be the starting vector x. The following picture describes the situation.

INSERT PICTURE HERE

Evidently the symmetric part of the matrix of our toy problem - the identity - has this property, where its coercivity constant is at most d. Moreover, note that the anti-symmetric part of our matrix fails to be coercive. Indeed, this matrix is a rotation matrix: it takes any vector x and rotates to a position orthogonal to itself, and hence $(Qx, x) = 0$ for any x. No cone will be able to contain the image of this rotation operator.

INSERT PICTURE HERE

We should also point out that coercivity and one-to-one-ness of an operator are closely related. Indeed, if an operator is coercive, then it is one-to-one. However, the other way around is false in general: the rotation operator is one-to-one (in fact fully invertible), yet fails to be coercive. In the eyes of coercivity, rotation and the 0-map behave the same way, which hints again to a possible failure while trying to construct a solver based on the "weak-form" (Ax, y).

Thus, we see that coercivity can be used as a proxy for testing invertibility of an operator, if you have this property, then you know you are "bounded away from zero." Moreover, the "amount of information" present in the coercivity constant is relevant. Consider for instance our original matrix:

$$x^T \left(d \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + a \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \right) x = d||x||^2$$

The coercivity constant is d and contains information only on the symmetric part of the operator while completely ignoring the anti-symmetric part. Thus, we are lead to believe that the more information we can "pack" on the coercivity constant, the more stable our numerical solver will be.

8 Stabilization: SUPG

Going back to our original system

$$Ax = b$$

and then consider the new system

$$MAx = Mb \quad \text{or} \quad M(Ax - b) = 0$$

where M is any invertible matrix. This can be thought as a "change of coordinates," where the solution of one system is the solution of the other an vice versa. If this new system with the M matrix were to be our stabilized numerical scheme, it would be called "consistent" if it had this property: solving the new system is the same as solving the original one.

In order to derive a good choice for M we follow the logic from the observations of the previous section. The issued arises from the mismatch between the information of the problem, present in the coefficients of the matrices, and the what the numerical scheme was able to "see." Specifically, the scheme picked up the information coming from the symmetric part of A, the identity, but ignored the anti-symmetric part - the rotation. Whenever most of the information of the problem lies on the anti-symmetric part, the case of a high Peclet number, the scheme produced a bad approximation.

Thankfully, a reversal of roles is easy to achieve. Rotations, much the imaginary number i, have the following cycle property

$$\begin{aligned} R(I) &= R \\ R(R) &= -I \\ R(-I) &= -R \\ R(-R) &= I \end{aligned}$$

Thus, choosing $M = R$ leads to the following

$$MA = R(dI + aR) = dR - aI$$

Now the coefficient d is associated with the anti-symmetric part, and a to the symmetric part. We should know check the connectivity of this operator. Let $x \in \mathbb{R}^2$ we have that

$$(MAx, x) = -a||x||^2$$

This almost look like good news, as we are now able to recover the a constant, which had previously been ignored. However, we can not claim that the operator is coercive, particularly because we don't know the sign of a. To fix this, we change to $M = -aR$ and compute again

$$(MAx, x) = a^2||x||^2$$

Now the operator has a coercivity constant related to a as opposed to d like it did before. This choice of M is known in the world of numerical PDES as **Streamlined Upwind Petrov-Galerkin** or **SUPG** for short. It is easy to check that using the weak form and solving on V_h as before we obtain

$$x_h = \begin{bmatrix} 1 \\ 0 \end{bmatrix} - \begin{bmatrix} d/a \\ 0 \end{bmatrix}$$

and the norm of the error is

$$||e_h|| = |d/a| = 1/Pe$$

Note that it is the inverse of the previous case! The error, instead of being equal to the Peclet number, it is equal to its inverse, thus the bigger the Peclet number, the smaller the error.

All in all, we have have found a way of dealing with the two cases. Whenever have a low Peclet number, we proceed as it was initially described, and whenever it is high, we "stabilize" it using the technique described. It is also worth noting that there are other choices of M that will achieve this effect. In general we may consider operators of the form

$$M = cI + dR$$

where choosing c and d lead to different schemes. In SUPG we have $c = 0$ and $d = -a$, but other options would lead to schemes like **Galerkin-Least Squares** and **Variational Multiscale** methods.

References